



IoT Laboratory Report

*Design and Implementation of an IoT System:
From Sensor Node to Gateway Integration*

Academic Year 2026

Authors:

Pietro Corti

Wassim Meguellati

Contents

1	General Introduction to the Internet of Things	3
2	Part 1 – IoT Node with ESP32 and BME680	5
2.1	Equipment	5
2.2	Hardware Connections	7
2.3	Code Description	7
2.3.1	Sub-step 1: BME680 Sensor Reading	7
2.3.2	Sub-step 2: Wi-Fi Access Point and HTTP Server	10
2.3.3	Sub-step 3: Combined Sensor Node with Web Dashboard	13
3	Part 2 – Gateway with Raspberry Pi and MQTT Broker	20
3.1	Equipment and Libraries	20
3.2	Operating System Preparation	21
3.3	First Boot and <code>raspi-config</code>	22
3.4	Mosquitto Installation and Test	23
3.5	Results	23
4	Part 3 – Complete IoT System Integration	25
4.1	Equipment	25
4.2	Hardware Connections	26
4.3	Broker Configuration for Remote Connections	26
4.4	Code Description	28
4.5	Results	30

5 Conclusions

34

Chapter 1

General Introduction to the Internet of Things

The Internet of Things (IoT) is a paradigm of computing in which everyday physical objects are equipped with sensors, processing units, and network connectivity, allowing them to collect, exchange, and act upon data without direct human intervention. Over the last decade, IoT has evolved from an academic concept to one of the cornerstones of modern digital infrastructure, supporting applications in smart homes, industrial automation, healthcare, agriculture, environmental monitoring, and smart cities.

A typical IoT system is structured in three logical layers:

- **Perception layer (nodes):** composed of microcontrollers and sensors that acquire physical measurements from the environment (temperature, humidity, pressure, gas concentration, etc.).
- **Network layer (gateway):** responsible for aggregating data from multiple nodes, often through lightweight protocols such as MQTT or HTTP, and forwarding them toward the upper layer.
- **Application layer:** where data are stored, processed, visualized, and used to support decisions or trigger automated actions.

The purpose of this laboratory is to gain a hands-on understanding of how these layers interact in practice. The work is divided into three progressive parts: first, the construction of a single IoT node based on the ESP32 microcontroller and the BME680 environmental sensor; second, the configuration of a gateway based on a Raspberry Pi running an MQTT broker; finally, the integration of both elements into a complete IoT system in which the node publishes sensor data to the broker and a client subscribes to receive them.

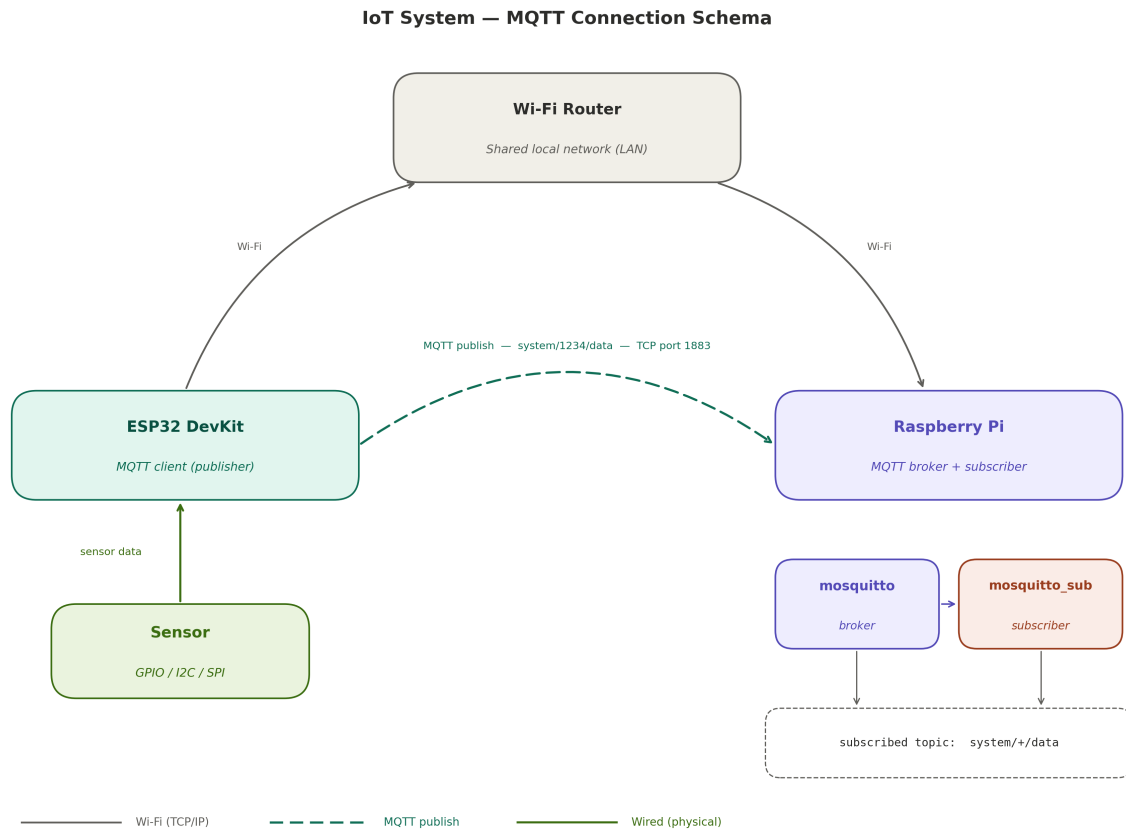


Figure 1.1: IoT System — MQTT Connection Schema of the laboratory setup

The data flow can be summarized as follows:

1. The BME680 sensor produces digital readings (temperature, humidity, pressure, gas resistance) and transmits them to the ESP32 over the SPI bus.
2. The ESP32 connects to the Wi-Fi access point, packages the readings into a JSON payload and publishes them to the MQTT broker.
3. The `mosquitto` broker on the Raspberry Pi receives the message and forwards it to all clients subscribed to the matching topic.
4. The `mosquitto_sub` client on the Raspberry Pi prints the received data on the console, completing the chain.

Chapter 2

Part 1 – IoT Node with ESP32 and BME680

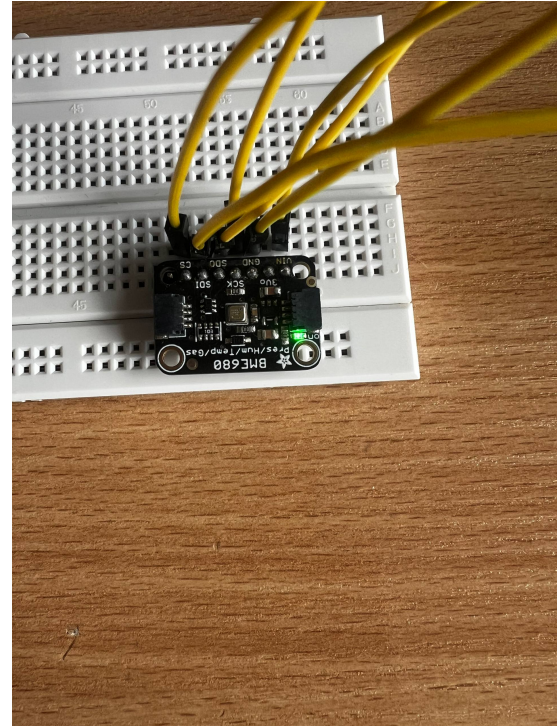
2.1 Equipment

The hardware and software used in the first part of the laboratory are the following:

- ESP32 DevKitC (DOIT ESP32 DEVKIT V1) microcontroller board.
- BME680 environmental sensor (temperature, humidity, pressure and gas) – Adafruit breakout board.
- Breadboard and jumper wires.
- USB cable (Micro-USB) for programming and powering the ESP32.
- Personal computer with the Arduino IDE installed.
- Adafruit BME680 Library for Arduino.



(a) ESP32 DevKitC (DOIT V1).



(b) Adafruit BME680 sensor on breadboard.

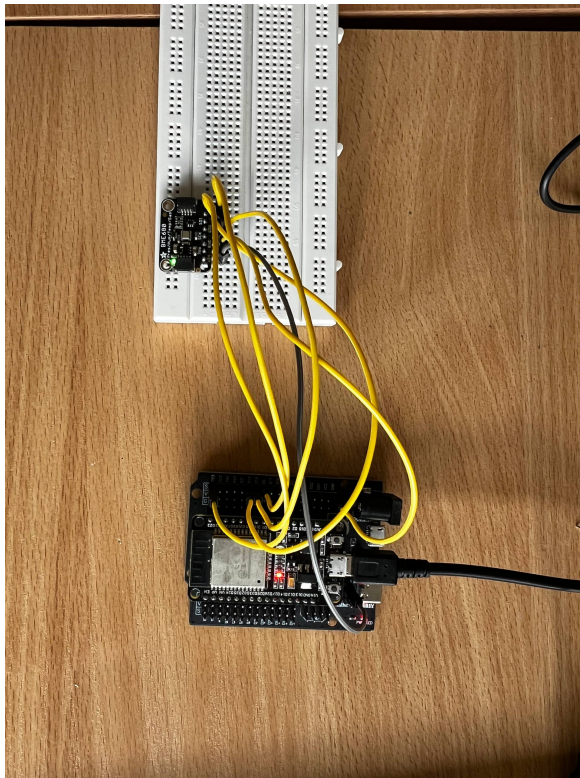
Figure 2.1: Main hardware components used in Part 1.

The connections between the ESP32 and the BME680 sensor follow the SPI scheme summarized in Table 2.1.

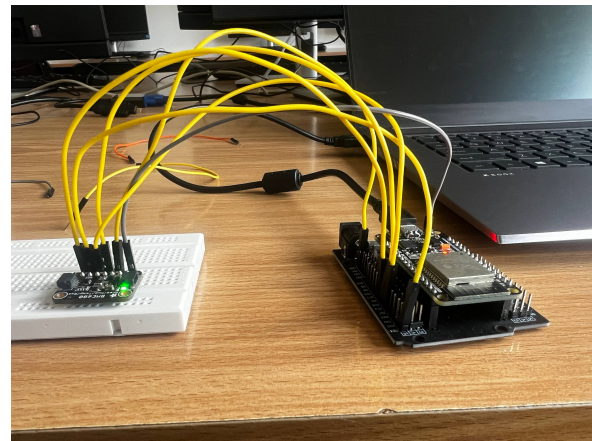
Table 2.1: Connection list between ESP32 and BME680.

ESP32 pin	BME680 pin
Vin	Vin
do not connect!	3Vo (leave unconnected)
GND	GND
D18	SCK
D19	SDO
D23	SDI
D5	CS

2.2 Hardware Connections



(a) Top view of the wiring.



(b) Alternative angle of the wiring.

Figure 2.2: Hardware connection between the ESP32 DevKit and the BME680 sensor on the breadboard. The yellow wires implement the SPI bus (SCK, SDO, SDI, CS), while the black/white wires carry power (Vin) and ground (GND).

2.3 Code Description

The firmware for Part 1 was developed in three incremental sub-steps. Each sub-step extends the previous one, allowing each block (sensor, Wi-Fi server, and their integration) to be tested independently before being merged into the final node firmware.

2.3.1 Sub-step 1: BME680 Sensor Reading

The first sub-step is built on top of the example `bme680test` provided by the Adafruit BME680 Library. The example was modified by:

- changing the definition of `BME_CS` to 5,
- commenting the line that uses the I2C constructor (`Adafruit_BME680 bme(&Wire);`),

- uncommenting the line that uses the SPI constructor with chip-select pin (`Adafruit_BME680 bme(BME_CS);`).

The code initializes the sensor over hardware SPI, configures oversampling and filtering options, and prints the four physical measurements (temperature, pressure, humidity, gas resistance) plus the estimated altitude on the Serial Monitor every two seconds.

```
1 #include <Wire.h>
2 #include <SPI.h>
3 #include <Adafruit_Sensor.h>
4 #include "Adafruit_BME680.h"
5
6 #define BME_SCK 13
7 #define BME_MISO 12
8 #define BME_MOSI 11
9 #define BME_CS 5
10
11 #define SEALEVELPRESSURE_HPA (1013.25)
12
13 //Adafruit_BME680 bme(&Wire); // I2C
14 //Adafruit_BME680 bme(&Wire1); // example of I2C on another bus
15 Adafruit_BME680 bme(BME_CS); // hardware SPI
16 //Adafruit_BME680 bme(BME_CS, BME_MOSI, BME_MISO, BME_SCK);
17
18 void setup() {
19   Serial.begin(9600);
20   while (!Serial);
21   Serial.println(F("BME680 test"));
22
23   if (!bme.begin()) {
24     Serial.println("Could not find a valid BME680 sensor, check wiring!");
25     while (1);
26   }
27
28   // Set up oversampling and filter initialization
29   bme.setTemperatureOversampling(BME680_OS_8X);
30   bme.setHumidityOversampling(BME680_OS_2X);
31   bme.setPressureOversampling(BME680_OS_4X);
32   bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
33   bme.setGasHeater(320, 150); // 320*C for 150 ms
34 }
35
36 void loop() {
37   if (! bme.performReading()) {
38     Serial.println("Failed to perform reading :(");
39     return;
```

```
40 }  
41 Serial.print("Temperature = ");  
42 Serial.print(bme.temperature);  
43 Serial.println(" *C");  
44  
45 Serial.print("Pressure = ");  
46 Serial.print(bme.pressure / 100.0);  
47 Serial.println(" hPa");  
48  
49 Serial.print("Humidity = ");  
50 Serial.print(bme.humidity);  
51 Serial.println(" %");  
52  
53 Serial.print("Gas = ");  
54 Serial.print(bme.gas_resistance / 1000.0);  
55 Serial.println(" KOhms");  
56  
57 Serial.print("Approx. Altitude = ");  
58 Serial.print(bme.readAltitude(SEALEVELPRESSURE_HPA));  
59 Serial.println(" m");  
60  
61 Serial.println();  
62 delay(2000);  
63 }
```

Listing 2.1: Sub-step 1 – BME680 sensor reading over SPI.

The Arduino IDE Serial Monitor confirmed that the BME680 sensor was correctly initialized and that all four measurements were available with stable values, as shown in Figure 2.3.

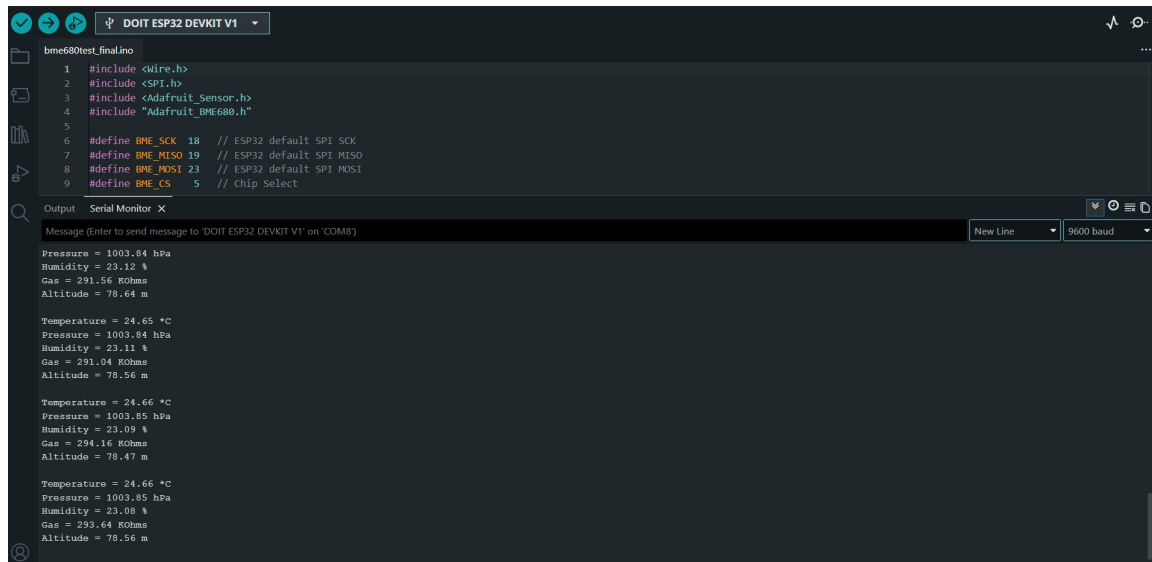


Figure 2.3: Sub-step 1: Serial Monitor output showing the BME680 readings (temperature, pressure, humidity, gas resistance and altitude).

2.3.2 Sub-step 2: Wi-Fi Access Point and HTTP Server

The second sub-step is based on the `SimpleWiFiServer` example from the ESP32 Wi-Fi library. Instead of connecting to an external access point, the ESP32 is configured to create its own Wi-Fi access point (ESP32-AP, password 12345678) and start a web server on port 80. The page returned by the server contains two hyperlinks (/H and /L) that allow the user to turn on or off an LED connected to pin 5 of the ESP32. This sub-step validates both the Wi-Fi soft-AP functionality and the HTTP request parsing.

```

1 #include <Arduino.h>
2 #include <WiFi.h>
3
4 const char *ssid      = "ESP32-AP";
5 const char *password  = "12345678"; // Min 8 characters
6
7 NetworkServer server(80);
8
9 void setup() {
10   Serial.begin(115200);
11   pinMode(5, OUTPUT);
12
13   delay(10);
14   Serial.println();
15
16   // Start Access Point
17   Serial.print("Starting Access Point: ");
18   Serial.println(ssid);
19

```

```
20  WiFi.softAP(ssid, password);
21
22  IPAddress IP = WiFi.softAPIP();
23  Serial.println("AP started!");
24  Serial.print("IP address: ");
25  Serial.println(IP);
26
27  server.begin();
28  Serial.println("Server started!");
29 }
30
31 void loop() {
32   NetworkClient client = server.accept();
33
34   if (client) {
35     Serial.println("New Client.");
36     String currentLine = "";
37
38     while (client.connected()) {
39       if (client.available()) {
40         char c = client.read();
41         Serial.write(c);
42
43         if (c == '\n') {
44           if (currentLine.length() == 0) {
45             client.println("HTTP/1.1 200 OK");
46             client.println("Content-type:text/html");
47             client.println();
48
49             client.print("Click <a href=\"/H\">here</a> to turn LED on  
pin 5 ON.<br>");
50             client.print("Click <a href=\"/L\">here</a> to turn LED on  
pin 5 OFF.<br>");
51
52             client.println();
53             break;
54           } else {
55             currentLine = "";
56           }
57         } else if (c != '\r') {
58           currentLine += c;
59         }
60
61         if (currentLine.endsWith("GET /H")) {
62           digitalWrite(5, HIGH);
63         }
64         if (currentLine.endsWith("GET /L")) {
```



```

65     digitalWrite(5, LOW);
66   }
67 }
68 }
69
70 client.stop();
71 Serial.println("Client Disconnected.");
72 }
73 }

```

Listing 2.2: Sub-step 2 – Wi-Fi access point and simple HTTP server.

The serial output of this sub-step (Figure 2.4) shows the ESP32 correctly accepting HTTP connections from a client, parsing the standard HTTP request headers, and serving the simple HTML page on the address 192.168.4.1 (Figure 2.5).

```

SimpleWiFiServer.ino
16 // -- Start Access Point
17 Serial.print("Starting Access Point: ");
18 Serial.println(ssid);
19
20 WiFi.softAP(ssid, password);
21
22 IPAddress IP = WiFi.softAPIP();
23 Serial.println("AP started!");
24 Serial.print("IP address: ");

```

Output Serial Monitor X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM8')

```

11:23:28.058 -> New Client.
11:23:28.090 -> GET / HTTP/1.1
11:23:28.123 -> Host: 192.168.4.1
11:23:28.123 -> Upgrade-Insecure-Requests: 1
11:23:28.123 -> Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
11:23:28.123 -> User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 26_3_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/147.0.7727.99 Mobile/15E148 Safari/604.1
11:23:28.123 -> Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
11:23:28.123 -> Accept-Encoding: gzip, deflate
11:23:28.123 -> Connection: keep-alive
11:23:28.162 ->
11:23:28.162 -> Client Disconnected.
11:23:28.162 -> New Client.
11:23:28.197 -> GET /favicon.ico HTTP/1.1
11:23:28.197 -> Host: 192.168.4.1
11:23:28.197 -> Connection: keep-alive
11:23:28.197 -> User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 26_3_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/147.0.7727.99 Mobile/15E148 Safari/604.1
11:23:28.197 -> Accept-Encoding: gzip, deflate
11:23:28.197 -> Accept-Language: en-US,en;q=0.9,en;q=0.8,fr;q=0.7
11:23:28.197 ->
11:23:28.197 -> Client Disconnected.

```

Ln 20, Col 47 DOIT ESP32 DEVKIT V1 on COM8

Figure 2.4: Sub-step 2: Serial Monitor showing the parsed HTTP request headers received by the ESP32.

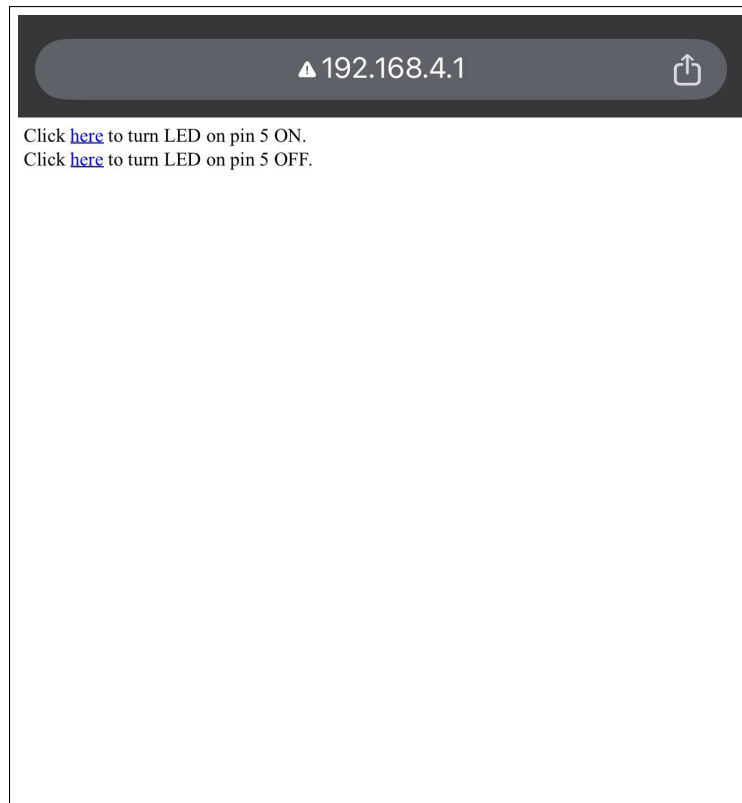


Figure 2.5: Sub-step 2: simple HTML page served by the ESP32 on 192.168.4.1.

2.3.3 Sub-step 3: Combined Sensor Node with Web Dashboard

In the final sub-step, the BME680 reading code from Sub-step 1 was merged with the Wi-Fi web server from Sub-step 2. The resulting firmware exposes a complete web dashboard on port 80: each HTTP request returns an HTML page containing the latest temperature, humidity, pressure, gas resistance and altitude readings, together with two buttons to control an LED. The page automatically refreshes every five seconds, providing a near real-time view of the environmental conditions. CSS styling is embedded directly in the HTML for a clean and responsive layout.

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <SPI.h>
4 #include <Adafruit_Sensor.h>
5 #include "Adafruit_BME680.h"
6 #include <WiFi.h>
7
8 // WiFi AP Credentials
9 const char *ssid      = "ESP32-BME680";
10 const char *password = "12345678";
11
12 // BME680 SPI Setup
```

```

13 #define BME_CS 5
14 #define SEALEVELPRESSURE_HPA (1013.25)
15 Adafruit_BME680 bme(BME_CS);
16
17 // LED Pin
18 #define LED_PIN 2    // Built-in LED on most ESP32 boards
19
20 // Web Server
21 NetworkServer server(80);
22
23 // Build HTML Page
24 String buildPage(bool ledState) {
25     if (!bme.performReading()) {
26         return "<h2>Failed to read BME680!</h2>";
27     }
28
29     String html = "<!DOCTYPE html><html><head>";
30     html += "<meta charset='UTF-8'>";
31     html += "<meta http-equiv='refresh' content='5'>";
32     html += "<title>ESP32 BME680</title>";
33     html += "<style>";
34     html += "    body { font-family: Arial; text-align: center; background:
35         #f0f0f0; margin: 0; padding: 20px; }";
36     html += "    h1 { color: #333; }";
37     html += "    .card { background: white; border-radius: 12px; padding: 20
38         px;
39         margin: 15px auto; max-width: 300px; box-shadow: 0
40         2px 8px rgba(0,0,0,0.1); }";
41     html += "    .value { font-size: 2em; font-weight: bold; color: #e67e22;
42         }";
43     html += "    .label { color: #888; font-size: 0.9em; margin-bottom: 5px;
44         }";
45     html += "    .btn { display: inline-block; padding: 12px 30px; margin: 8
46         px;
47         border-radius: 8px; text-decoration: none; font-size
48         : 1em; font-weight: bold; }";
49     html += "    .btn-on { background: #2ecc71; color: white; }";
50     html += "    .btn-off { background: #e74c3c; color: white; }";
51     html += "    .led-status { font-size: 1.1em; margin-top: 10px; }";
52     html += "</style></head><body>";
53
54     html += "<h1>ESP32 BME680 Monitor</h1>";
55
56     // Sensor Cards
57     html += "<div class='card'><div class='label'>Temperature</div>";
58     html += "<div class='value'>" + String(bme.temperature, 1) + " C</div
59         ></div>";

```

```

52
53   html += "<div class='card'><div class='label'>Humidity</div>";
54   html += "<div class='value'>" + String(bme.humidity, 1) + " %</div></div>";
55
56   html += "<div class='card'><div class='label'>Pressure</div>";
57   html += "<div class='value'>" + String(bme.pressure / 100.0, 1) + " hPa</div></div>";
58
59   html += "<div class='card'><div class='label'>Gas Resistance</div>";
60   html += "<div class='value'>" + String(bme.gas_resistance / 1000.0, 1) + " KOhm</div></div>";
61
62   html += "<div class='card'><div class='label'>Altitude</div>";
63   html += "<div class='value'>" + String(bme.readAltitude(
        SEALEVELPRESSURE_HPA), 1) + " m</div></div>";
64
65   // LED Control Card
66   html += "<div class='card'>";
67   html += "<div class='label'>LED Control</div>";
68   html += "<div class='led-status'>Status: <strong>" + String(ledState ?
        "ON" : "OFF") + "</strong></div>";
69   html += "<br>";
70   html += "<a href='/H' class='btn btn-on'>Turn ON</a>";
71   html += "<a href='/L' class='btn btn-off'>Turn OFF</a>";
72   html += "</div>";
73
74   html += "<p style='color:#aaa; font-size:0.85em;'>Auto-refreshes every 5 seconds</p>";
75   html += "</body></html>";
76
77   return html;
78 }
79
80 void setup() {
81   Serial.begin(9600);
82   delay(2000);
83
84   pinMode(LED_PIN, OUTPUT);
85   digitalWrite(LED_PIN, LOW);
86
87   // BME680 Init
88   if (!bme.begin()) {
89     Serial.println("BME680 not found! Check wiring.");
90     while (1);
91   }
92   bme.setTemperatureOversampling(BME680_OS_8X);

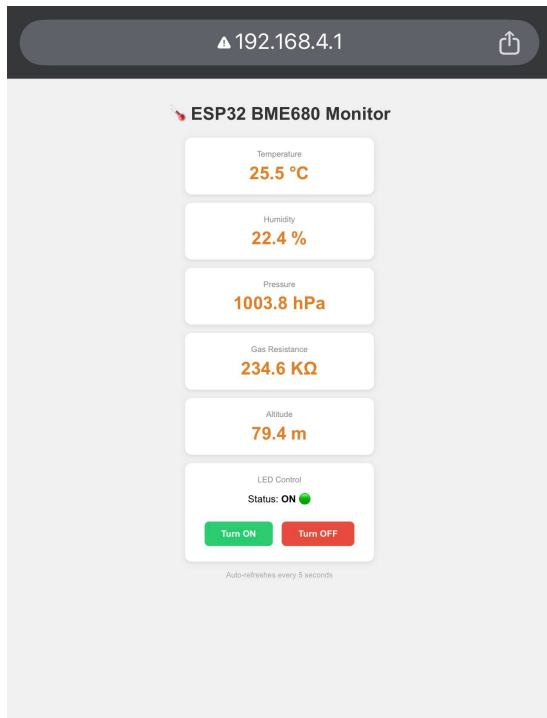
```

```
93  bme.setHumidityOversampling(BME680_OS_2X);
94  bme.setPressureOversampling(BME680_OS_4X);
95  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
96  bme.setGasHeater(320, 150);
97  Serial.println("BME680 ready!");
98
99  // Start Access Point
100  WiFi.softAP(ssid, password);
101  IPAddress IP = WiFi.softAPIP();
102  Serial.print("AP started! SSID: ");
103  Serial.println(ssid);
104  Serial.print("Open browser at: http://");
105  Serial.println(IP);
106
107  server.begin();
108  Serial.println("Server started!");
109 }
110
111 bool ledState = false;
112
113 void loop() {
114     NetworkClient client = server.accept();
115
116     if (client) {
117         Serial.println("New Client.");
118         String currentLine = "";
119
120         while (client.connected()) {
121             if (client.available()) {
122                 char c = client.read();
123                 Serial.write(c);
124
125                 if (c == '\n') {
126                     if (currentLine.length() == 0) {
127                         // Send HTTP Response
128                         client.println("HTTP/1.1 200 OK");
129                         client.println("Content-type:text/html");
130                         client.println();
131                         client.print(buildPage(ledState));
132                         client.println();
133                         break;
134                     } else {
135                         currentLine = "";
136                     }
137                 } else if (c != '\r') {
138                     currentLine += c;
139                 }
```

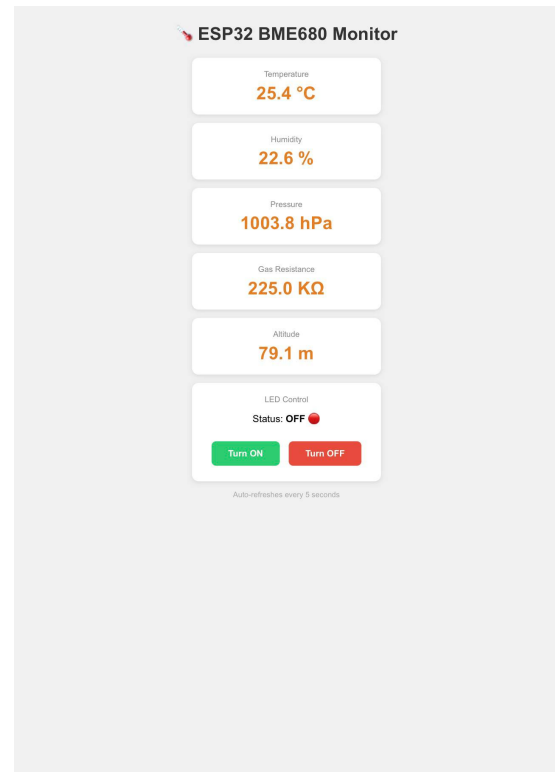
```
140
141     // LED Control
142     if (currentLine.endsWith("GET /H")) {
143         digitalWrite(LED_PIN, HIGH);
144         ledState = true;
145         Serial.println("LED ON");
146     }
147     if (currentLine.endsWith("GET /L")) {
148         digitalWrite(LED_PIN, LOW);
149         ledState = false;
150         Serial.println("LED OFF");
151     }
152 }
153 }
154
155     client.stop();
156     Serial.println("Client Disconnected.");
157 }
158 }
```

Listing 2.3: Sub-step 3 – Combined BME680 sensor + Wi-Fi dashboard with LED control.

The final integrated dashboard is presented in Figures 2.6a and 2.6b: a styled web interface displays the live BME680 readings (temperature near 25.5 °C, pressure near 1003.8 hPa, gas resistance around 225–234 kΩ) and the LED control buttons, with a status indicator that updates accordingly. The corresponding hardware behaviour is documented in Figures 2.7a and 2.7b: the on-board LED on pin 2 is correctly driven by the buttons of the dashboard, providing a visible feedback that confirms the bidirectional behaviour of the web server.

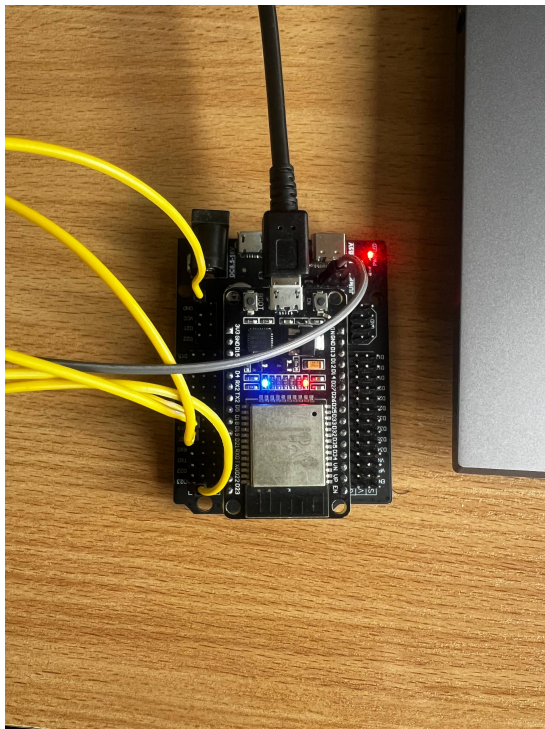


(a) Dashboard with LED status ON.

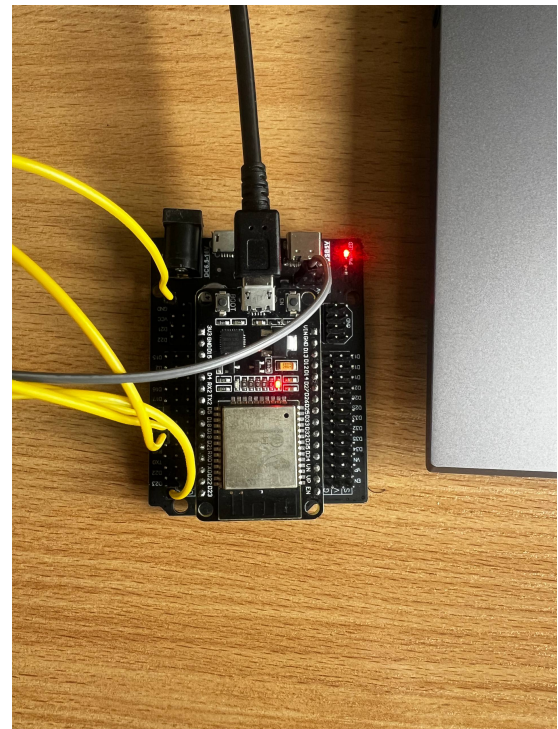


(b) Dashboard with LED status OFF.

Figure 2.6: Sub-step 3: final web dashboard served by the ESP32, showing live BME680 readings and LED control.



(a) ESP32 with LED ON (blue LED lit on pin 2).



(b) ESP32 with LED OFF (only the power LED is lit).

Figure 2.7: Hardware feedback on the ESP32 board, controlled from the web dashboard.

The readings remained stable throughout the experiment, and the dashboard responded to button clicks within a few hundred milliseconds, confirming that the ESP32 is more than capable of handling the combined task of sensor sampling and HTTP serving.

Chapter 3

Part 2 – Gateway with Raspberry Pi and MQTT Broker

3.1 Equipment and Libraries

The gateway is built using the following components:

- Raspberry Pi single-board computer with microSD card.
- Power supply, HDMI cable, monitor, keyboard and mouse.
- Ethernet cable or Wi-Fi connection to the local router.
- Raspberry Pi OS (clean installation prepared with Raspberry Pi Imager).
- Mosquitto MQTT broker and `mosquitto-clients` command-line tools.

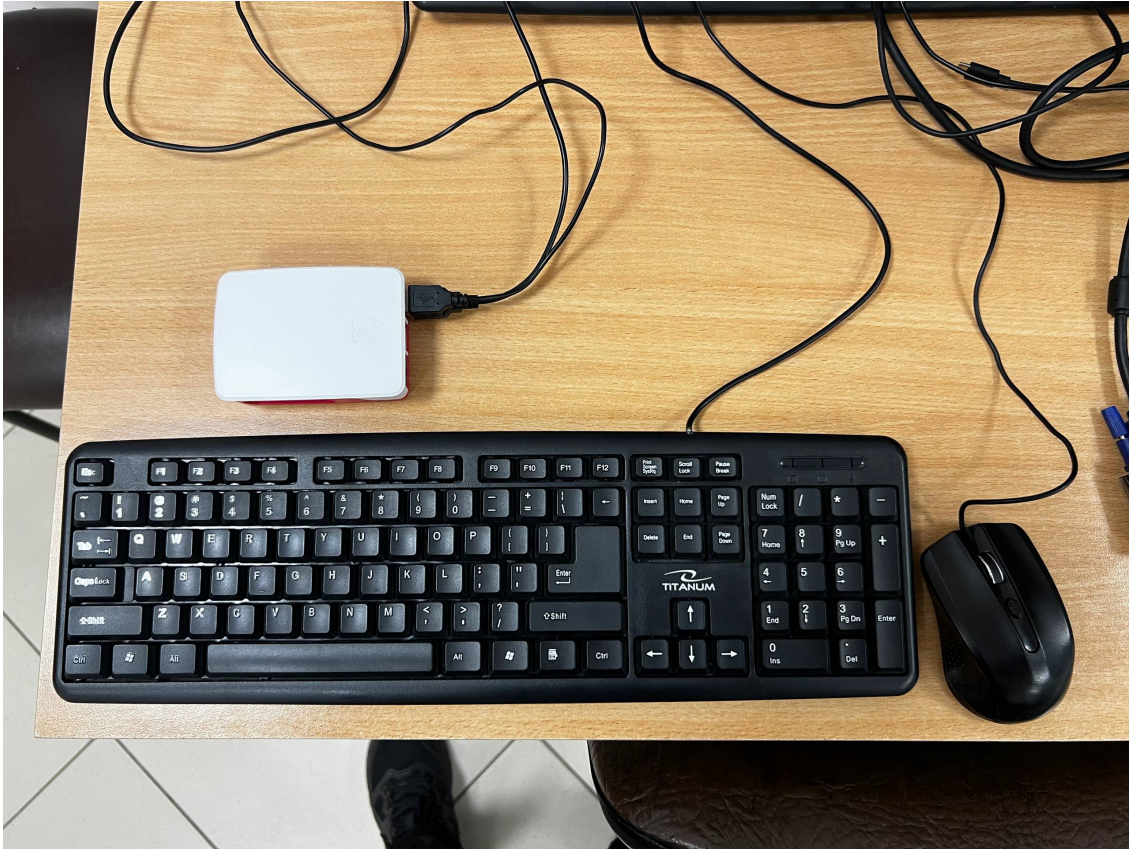


Figure 3.1: Raspberry Pi Full Setup

3.2 Operating System Preparation

The first step in building the gateway was to prepare a clean installation of Raspberry Pi OS using the official *Raspberry Pi Imager* tool, downloaded from the Raspberry Pi website (Figure 3.2a). During the configuration step of the Imager, the username `pi`, the password `raspberry` and the SSH access were set in advance, so that the Pi could be reached remotely after the first boot (We still programmed from inside the Raspberry Pi interface). The disk image was then written to the microSD card (Figure 3.2b).

Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install Raspberry Pi OS and other operating systems to a microSD card, ready to use with your Raspberry Pi.

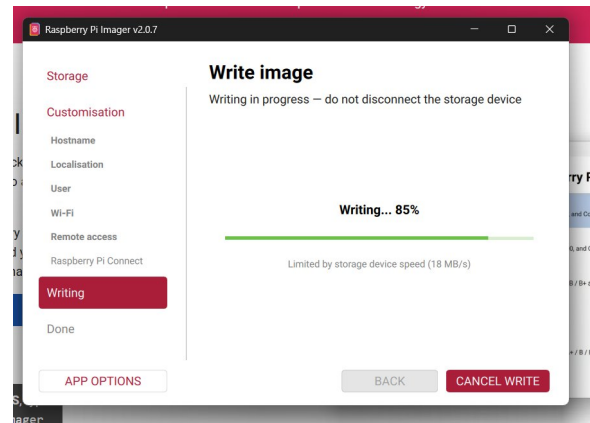
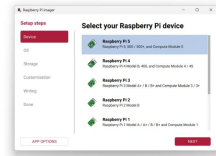
Download and install Raspberry Pi Imager on a computer with an SD card reader. Insert the microSD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

[Download for Windows](#) [Read the documentation](#)

[Download for macOS](#)

[Download for Linux \(x86_64\)](#)

To install on Raspberry Pi OS, type
sudo apt install rpi-imager
into a terminal window



(a) Raspberry Pi Imager download page.

(b) Imager flashing the microSD card.

Figure 3.2: Preparation of the Raspberry Pi OS installation through Raspberry Pi Imager.

3.3 First Boot and raspi-config

After inserting the microSD card and powering the Raspberry Pi, the system was further configured through the **raspi-config** utility, which allows the user to change low-level settings such as locale, timezone, interfaces and boot behaviour. In particular, the device was configured to boot to the text console (System Options → Boot / Auto Login → Console) as required by the laboratory specification.

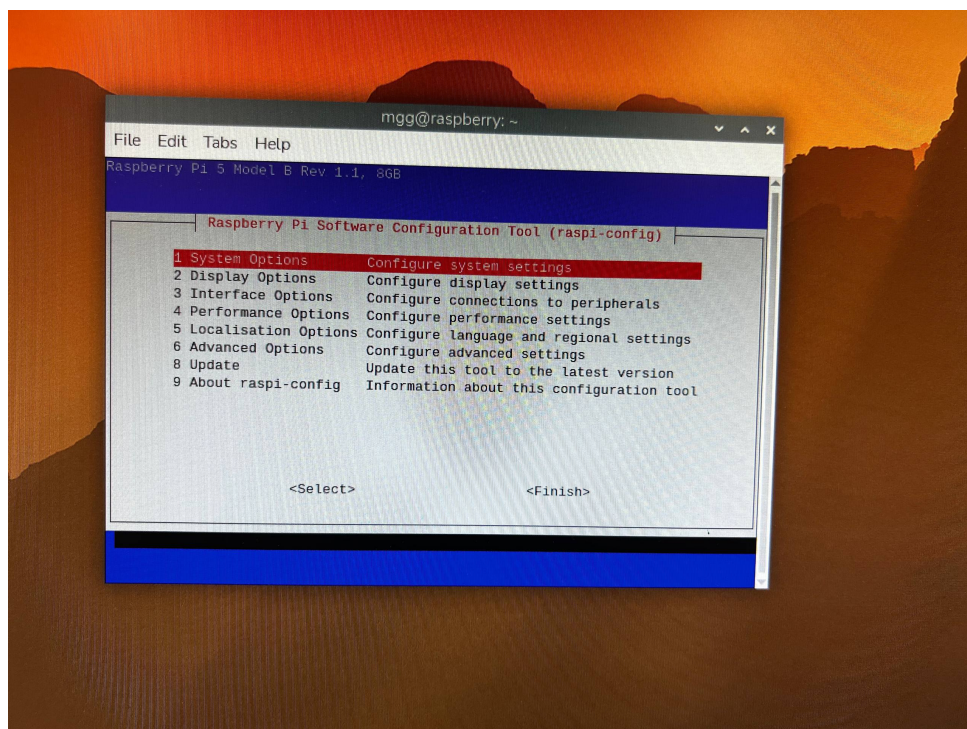


Figure 3.3: The **raspi-config** utility used to finalize the Raspberry Pi setup after the first boot.

3.4 Mosquitto Installation and Test

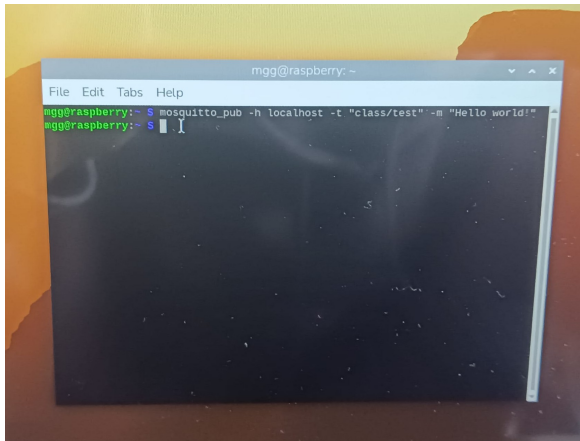
Once the operating system was up and running, the `mosquitto` broker and the command-line clients (`mosquitto-clients`) were installed from the official Raspberry Pi OS package repositories. The broker was then enabled to start automatically at boot, and a basic publish/subscribe test was performed locally on the Pi using two terminals: one running `mosquitto_sub` (subscribed to the topic `class/test`) and the other running `mosquitto_pub` (publishing the message "Hello world!" to the same topic). The relevant commands are reported below.

```
1 # Update package lists and install Mosquitto broker + clients
2 sudo apt update
3 sudo apt install mosquitto mosquitto-clients
4
5 # Enable Mosquitto to start automatically at boot
6 sudo systemctl enable mosquitto
7 sudo systemctl start mosquitto
8
9 # Check the broker status
10 sudo systemctl status mosquitto
11
12 # Subscribe to a test topic (terminal 1)
13 mosquitto_sub -h localhost -t class/test
14
15 # Publish a test message (terminal 2)
16 mosquitto_pub -h localhost -t class/test -m "Hello world!"
```

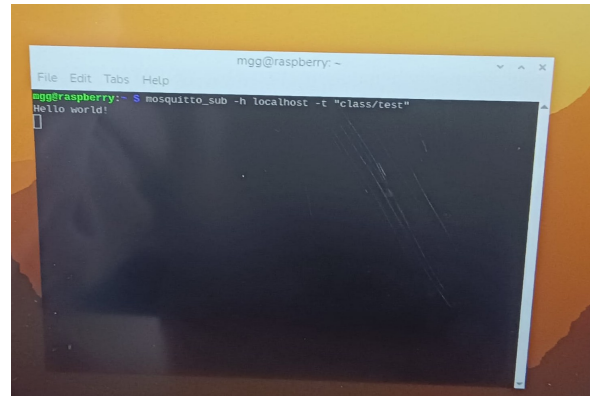
Listing 3.1: Mosquitto installation and basic test commands.

3.5 Results

The publish/subscribe mechanism worked at the first attempt: the message published with `mosquitto_pub` (Figure 3.4a) was received instantly by the `mosquitto_sub` client subscribed to the same topic, which printed it on the console (Figure 3.4b). This local test confirmed that the broker was correctly installed, running and capable of routing messages.



(a) mosquitto_pub publishing the test message.



(b) mosquitto_sub receiving the message.

Figure 3.4: Local validation of the Mosquitto broker on the Raspberry Pi using the topic class/test.

Chapter 4

Part 3 – Complete IoT System Integration

4.1 Equipment

The third part combines the equipment used in the previous two parts:

- ESP32 DevKit board with the BME680 sensor (from Part 1).
- Raspberry Pi running the Mosquitto MQTT broker (from Part 2).
- Wireless router providing the local Wi-Fi network.
- Personal computer for programming the ESP32 and accessing the Raspberry Pi.
- Arduino library `PubSubClient` for the MQTT client side on the ESP32.

The configuration requirements for this part are:

- The Raspberry Pi must be connected to the wireless router.
- The MQTT broker on the Raspberry Pi must be running and accept connections from external devices.
- The MQTT client on the Raspberry Pi must subscribe to the topic `system/+/data` (or, more specifically, `system/1234/data`).
- The ESP32 must be connected to the same wireless router and to the broker on the Raspberry Pi.
- The ESP32 must publish messages to the topic `system/1234/data`, containing real BME680 sensor data formatted as a JSON object.

4.2 Hardware Connections

The wiring of the ESP32 and the BME680 sensor is identical to Part 1; the only difference at the hardware level is that the Raspberry Pi gateway is now part of the same Wi-Fi network as the ESP32. Figure 4.1 shows the complete experimental setup with the Raspberry Pi (in its red/white case), the ESP32 + BME680 node on the breadboard, and the development workstation used to monitor both ends of the chain.



Figure 4.1: Complete IoT system setup: Raspberry Pi gateway (centre), ESP32 + BME680 node on breadboard (right), monitor and keyboard (left).

4.3 Broker Configuration for Remote Connections

To allow remote clients (in this case, the ESP32 connected over Wi-Fi) to connect to the Mosquitto broker on the Raspberry Pi, the configuration file `/etc/mosquitto/mosquitto.conf` was modified by adding the lines shown in Listing 4.1. The directive `listener 1883` opens the standard MQTT port on all interfaces, while `allow_anonymous true` disables authentication for this educational setup. Figure 4.2 shows the file as edited with GNU nano on the Raspberry Pi.

```
1 # Place your local configuration in /etc/mosquitto/conf.d/
```

```
2 #
3 # A full description of the configuration file is at
4 # /usr/share/doc/mosquitto/examples/mosquitto.conf
5
6 #pid_file /run/mosquitto/mosquitto.pid
7
8 persistence true
9 persistence_location /var/lib/mosquitto/
10
11 log_dest file /var/log/mosquitto/mosquitto.log
12
13 include_dir /etc/mosquitto/conf.d
14 listener 1883
15 allow_anonymous true
```

Listing 4.1: Mosquitto configuration file (/etc/mosquitto/mosquitto.conf).

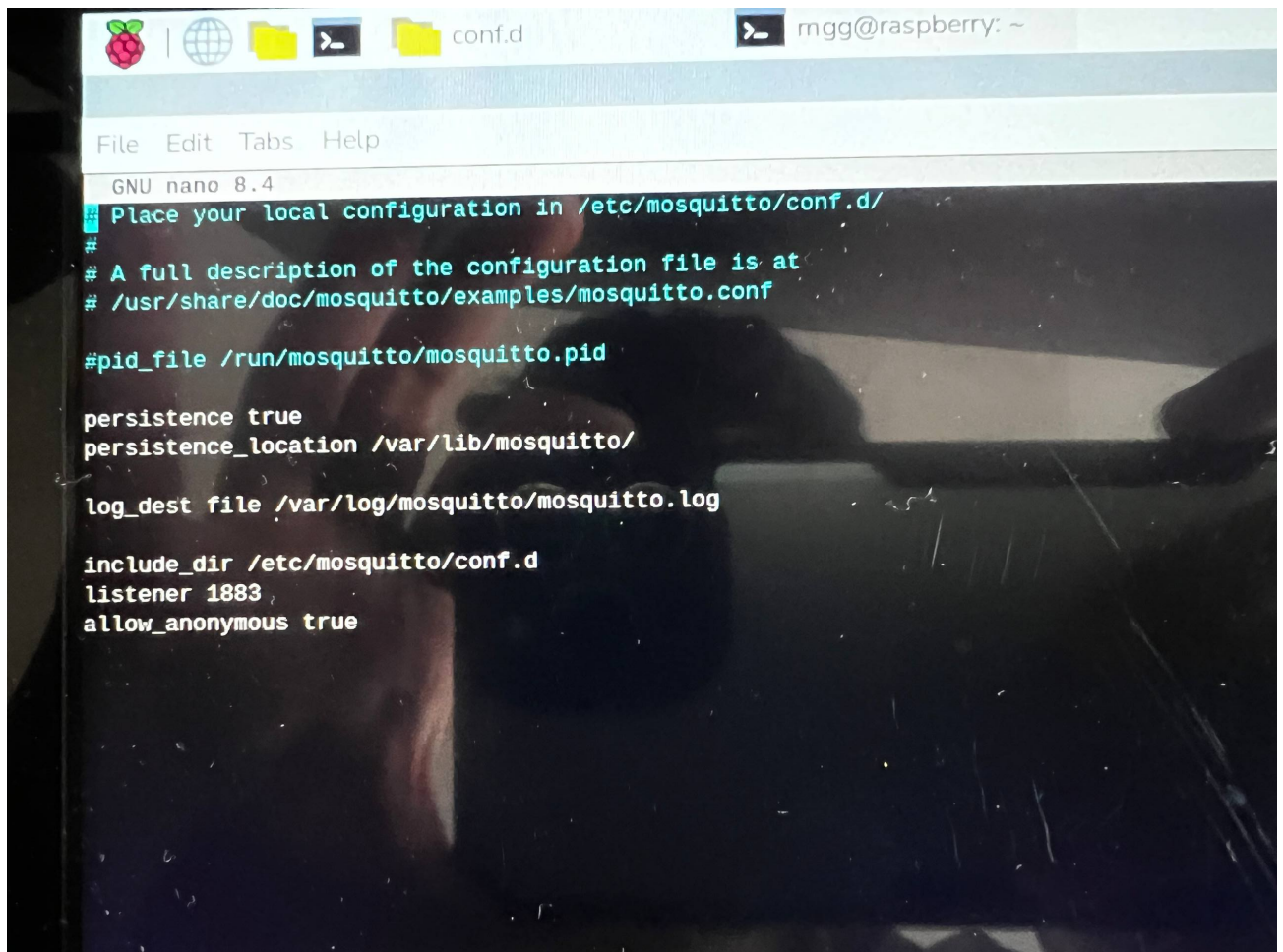


Figure 4.2: Mosquitto configuration file edited with GNU nano on the Raspberry Pi.

4.4 Code Description

The firmware running on the ESP32 in this part is a re-elaboration of the one developed in Part 1: instead of exposing the sensor data through HTTP, the node connects to the local Wi-Fi network as a client (no longer as an access point), then connects to the Mosquitto broker on the Raspberry Pi and publishes the readings on the topic `system/1234/data`. The payload is built as a JSON object containing all five quantities (temperature, pressure, humidity, gas resistance, altitude) and sent every two seconds. On the gateway side, the `mosquitto_sub` client is used to subscribe to the topic and print every received message on the terminal.

```
1 #include <Wire.h>
2 #include <SPI.h>
3 #include <Adafruit_Sensor.h>
4 #include "Adafruit_BME680.h"
5 #include <WiFi.h>
6 #include <PubSubClient.h>
7
8 // --- WiFi & MQTT config ---
9 const char* ssid      = "Pietro";
10 const char* password = "12345678ab";
11 const char* mqtt_server = "192.168.188.33"; // RPi's IP address
12 const char* mqtt_topic  = "system/1234/data";
13
14 // --- BME680 SPI pins ---
15 #define BME_SCK  13
16 #define BME_MISO 12
17 #define BME_MOSI 11
18 #define BME_CS   5
19 #define SEALEVELPRESSURE_HPA (1013.25)
20
21 Adafruit_BME680 bme(BME_CS);
22 WiFiClient espClient;
23 PubSubClient client(espClient);
24
25 // --- Connect to WiFi ---
26 void setup_wifi() {
27   delay(10);
28   Serial.print("Connecting to ");
29   Serial.println(ssid);
30   WiFi.begin(ssid, password);
31   while (WiFi.status() != WL_CONNECTED) {
32     delay(500);
33     Serial.print(".");
34   }
```

```
35 Serial.println("\nWiFi connected. IP: ");
36 Serial.println(WiFi.localIP());
37 }
38
39 // --- Reconnect to MQTT broker if disconnected ---
40 void reconnect() {
41     while (!client.connected()) {
42         Serial.print("Connecting to MQTT broker...");
43         if (client.connect("ESP32Client")) { // unique client ID
44             Serial.println("connected!");
45         } else {
46             Serial.print("failed, rc=");
47             Serial.print(client.state());
48             Serial.println(" -- retrying in 5s");
49             delay(5000);
50         }
51     }
52 }
53
54 void setup() {
55     Serial.begin(115200);
56     setup_wifi();
57     client.setServer(mqtt_server, 1883);
58
59     Serial.println("BME680 test");
60     if (!bme.begin()) {
61         Serial.println("Could not find a valid BME680 sensor, check wiring!");
62     };
63     while (1);
64
65     bme.setTemperatureOversampling(BME680_OS_8X);
66     bme.setHumidityOversampling(BME680_OS_2X);
67     bme.setPressureOversampling(BME680_OS_4X);
68     bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
69     bme.setGasHeater(320, 150);
70 }
71
72 void loop() {
73     if (!client.connected()) reconnect();
74     client.loop();
75
76     if (!bme.performReading()) {
77         Serial.println("Failed to perform reading :(");
78         return;
79     }
80 }
```

```
81 // Build a JSON payload
82 char payload[200];
83 snprintf(payload, sizeof(payload),
84  "{\"temperature\":%.2f,\"pressure\":%.2f,\"humidity\":%.2f,\"gas
85  \":%.2f,\"altitude\":%.2f}\",
86  bme.temperature,
87  bme.pressure / 100.0,
88  bme.humidity,
89  bme.gas_resistance / 1000.0,
90  bme.readAltitude(SEALEVELPRESSURE_HPA)
91 );
92 Serial.print("Publishing: ");
93 Serial.println(payload);
94 client.publish(mqtt_topic, payload);
95
96 delay(2000);
97 }
```

Listing 4.2: ESP32 firmware: BME680 readings published over MQTT to the Raspberry Pi broker.

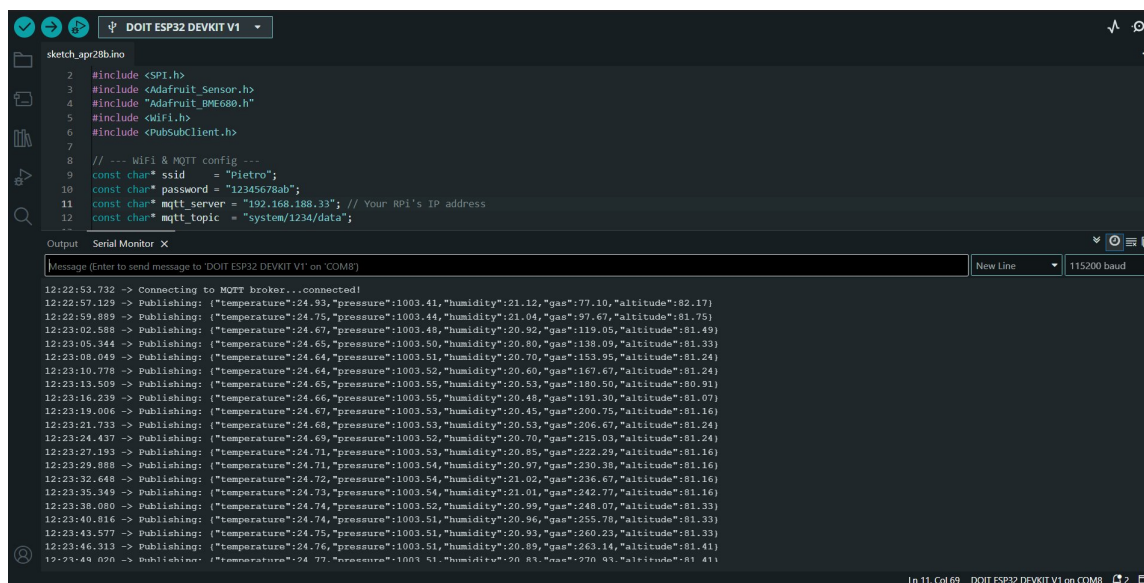
On the Raspberry Pi side, the subscription to the topic is performed with the following one-line command:

```
1 mosquitto_sub -h localhost -t "system/1234/data"
```

Listing 4.3: Raspberry Pi MQTT subscriber command.

4.5 Results

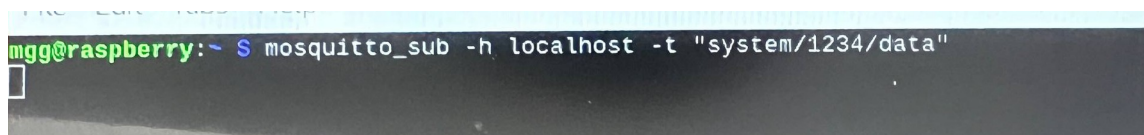
The ESP32 successfully connected to the local Wi-Fi network and to the Mosquitto broker on the Raspberry Pi at the first attempt. As shown in Figure 4.3, the Arduino IDE Serial Monitor confirms a stable MQTT connection (**Connecting to MQTT broker...connected!**) and a continuous stream of **Publishing:** messages, each containing a fresh JSON payload with the five environmental quantities sampled every two seconds.



The screenshot shows the Arduino IDE Serial Monitor window. The top part displays the sketch code for `sketch_apr28b.ino`, which includes headers for SPI, Adafruit BME680, WiFi, and PubSubClient. It defines MQTT configuration variables like `ssid`, `password`, `mqtt_server`, and `mqtt_topic`. The bottom part shows the serial output, where the ESP32 connects to the MQTT broker and publishes a series of JSON payloads. Each payload contains sensor data: temperature, pressure, humidity, gas, and altitude. The messages are timestamped and show the progression of data being sent to the broker.

Figure 4.3: Arduino IDE Serial Monitor showing the ESP32 publishing JSON payloads to the broker.

On the Raspberry Pi side, the `mosquitto_sub` client (Figure 4.4) received the same JSON messages in real time, as shown in Figure 4.5. The two streams are perfectly aligned in content (same temperature, pressure, humidity, gas resistance and altitude values) and only differ by the network propagation time, which is in the order of a few milliseconds on the local network. This proves that the full IoT pipeline, from the BME680 sensor up to the gateway terminal, is correctly working end-to-end.



The screenshot shows a terminal window on a Raspberry Pi. The prompt is `magg@raspberrypi:~`. The command entered is `S mosquitto_sub -h localhost -t "system/1234/data"`. The terminal is dark with green text, and the command is highlighted in yellow.

Figure 4.4: `mosquitto_sub` command issued on the Raspberry Pi terminal to listen on the topic `system/1234/data`.


```

File Edit Tabs Help
mgg@raspberrypi:~$ mosquitto_sub -h localhost -t "system/1234/data"
{"temperature":24.93,"pressure":1003.41,"humidity":21.12,"gas":77.10,"altitude":82.17}
{"temperature":24.75,"pressure":1003.44,"humidity":21.04,"gas":97.67,"altitude":81.75}
{"temperature":24.67,"pressure":1003.48,"humidity":20.92,"gas":119.05,"altitude":81.49}
{"temperature":24.65,"pressure":1003.50,"humidity":20.80,"gas":138.09,"altitude":81.33}
{"temperature":24.64,"pressure":1003.51,"humidity":20.70,"gas":153.95,"altitude":81.24}
{"temperature":24.64,"pressure":1003.52,"humidity":20.60,"gas":167.67,"altitude":81.24}
{"temperature":24.65,"pressure":1003.55,"humidity":20.53,"gas":180.50,"altitude":80.91}
{"temperature":24.66,"pressure":1003.55,"humidity":20.48,"gas":191.30,"altitude":81.07}
{"temperature":24.67,"pressure":1003.53,"humidity":20.45,"gas":200.75,"altitude":81.16}
{"temperature":24.68,"pressure":1003.53,"humidity":20.53,"gas":206.67,"altitude":81.24}
{"temperature":24.69,"pressure":1003.52,"humidity":20.70,"gas":215.03,"altitude":81.24}
{"temperature":24.71,"pressure":1003.53,"humidity":20.85,"gas":222.29,"altitude":81.16}
{"temperature":24.71,"pressure":1003.54,"humidity":20.97,"gas":230.38,"altitude":81.16}
{"temperature":24.72,"pressure":1003.54,"humidity":21.02,"gas":236.67,"altitude":81.16}
{"temperature":24.73,"pressure":1003.54,"humidity":21.01,"gas":242.77,"altitude":81.16}
{"temperature":24.74,"pressure":1003.52,"humidity":20.99,"gas":248.07,"altitude":81.33}
{"temperature":24.74,"pressure":1003.51,"humidity":20.96,"gas":255.78,"altitude":81.33}
{"temperature":24.75,"pressure":1003.51,"humidity":20.93,"gas":260.23,"altitude":81.33}
{"temperature":24.76,"pressure":1003.51,"humidity":20.89,"gas":263.14,"altitude":81.41}
{"temperature":24.77,"pressure":1003.51,"humidity":20.83,"gas":270.93,"altitude":81.41}
{"temperature":24.78,"pressure":1003.52,"humidity":20.78,"gas":273.63,"altitude":81.07}
{"temperature":24.79,"pressure":1003.56,"humidity":20.79,"gas":278.73,"altitude":80.74}
{"temperature":24.79,"pressure":1003.57,"humidity":20.78,"gas":280.87,"altitude":80.91}
{"temperature":24.80,"pressure":1003.55,"humidity":20.73,"gas":284.75,"altitude":81.16}
{"temperature":24.81,"pressure":1003.52,"humidity":20.70,"gas":287.49,"altitude":81.41}
{"temperature":24.82,"pressure":1003.51,"humidity":20.65,"gas":290.53,"altitude":81.49}
{"temperature":24.82,"pressure":1003.50,"humidity":20.61,"gas":296.81,"altitude":81.58}
{"temperature":24.83,"pressure":1003.49,"humidity":20.53,"gas":298.16,"altitude":81.58}
{"temperature":24.83,"pressure":1003.49,"humidity":20.46,"gas":301.71,"altitude":81.58}
{"temperature":24.84,"pressure":1003.50,"humidity":20.38,"gas":303.94,"altitude":81.49}
{"temperature":24.84,"pressure":1003.50,"humidity":20.32,"gas":305.63,"altitude":81.49}
{"temperature":24.85,"pressure":1003.50,"humidity":20.29,"gas":308.20,"altitude":81.49}
{"temperature":24.85,"pressure":1003.50,"humidity":20.25,"gas":310.53,"altitude":81.49}
{"temperature":24.85,"pressure":1003.50,"humidity":20.22,"gas":312.29,"altitude":81.49}
{"temperature":24.85,"pressure":1003.50,"humidity":20.20,"gas":315.58,"altitude":81.49}
{"temperature":24.86,"pressure":1003.51,"humidity":20.18,"gas":317.10,"altitude":81.33}
{"temperature":24.86,"pressure":1003.51,"humidity":20.14,"gas":318.33,"altitude":81.41}
{"temperature":24.86,"pressure":1003.50,"humidity":20.12,"gas":319.88,"altitude":81.58}
{"temperature":24.85,"pressure":1003.50,"humidity":20.07,"gas":323.01,"altitude":81.49}
{"temperature":24.86,"pressure":1003.50,"humidity":20.06,"gas":324.29,"altitude":81.49}
{"temperature":24.86,"pressure":1003.49,"humidity":20.03,"gas":323.65,"altitude":81.58}
{"temperature":24.86,"pressure":1003.49,"humidity":20.01,"gas":326.54,"altitude":81.58}
{"temperature":24.87,"pressure":1003.50,"humidity":20.00,"gas":328.49,"altitude":81.49}
{"temperature":24.87,"pressure":1003.50,"humidity":20.00,"gas":328.49,"altitude":81.58}
{"temperature":24.87,"pressure":1003.48,"humidity":19.99,"gas":330.80,"altitude":81.75}
{"temperature":24.87,"pressure":1003.48,"humidity":19.98,"gas":331.47,"altitude":81.58}
{"temperature":24.87,"pressure":1003.51,"humidity":19.98,"gas":333.48,"altitude":81.33}

```

Figure 4.5: Sensor data received on the Raspberry Pi by the `mosquitto_sub` client subscribed to `system/1234/data`.

A representative excerpt of the received messages is reported below to show the structure of the published payload and the typical magnitude of the readings (temperature near 24.8°C, pressure near 1003.5 hPa, humidity around 20%, gas resistance progressively increasing as the gas-heating element of the BME680 stabilizes, altitude near 81 m):

```

1 {"temperature":24.93,"pressure":1003.41,"humidity":21.12,"gas":77.10,"
   altitude":82.17}
2 {"temperature":24.75,"pressure":1003.44,"humidity":21.04,"gas":97.67,"
   altitude":81.75}

```

```
3 {"temperature":24.67,"pressure":1003.48,"humidity":20.92,"gas":119.05,"
   altitude":81.49}
4 {"temperature":24.65,"pressure":1003.50,"humidity":20.80,"gas":138.09,"
   altitude":81.33}
5 {"temperature":24.64,"pressure":1003.51,"humidity":20.70,"gas":153.95,"
   altitude":81.24}
6 ...
7 {"temperature":24.87,"pressure":1003.51,"humidity":19.98,"gas":333.48,"
   altitude":81.33}
```

Listing 4.4: Example of MQTT messages received by `mosquitto_sub`.

The connection remained stable for several minutes of continuous operation, with no message loss observed on the subscriber side and an end-to-end latency low enough to consider the system suitable for real-time environmental monitoring.

Chapter 5

Conclusions

The laboratory provided a practical experience of the design and implementation of a complete Internet of Things system, starting from the basic building blocks and progressively integrating them into a single architecture. In Part 1, an ESP32 microcontroller was successfully interfaced with a BME680 environmental sensor and configured to expose the readings through HTTP, demonstrating how a microcontroller can act as a small web server. The development was carried out in three incremental sub-steps – sensor reading, Wi-Fi access point with HTTP server, and the integrated dashboard – which made debugging and validation considerably easier. In Part 2, a Raspberry Pi was configured as a gateway running the Mosquitto MQTT broker, and the publish/subscribe mechanism was validated locally with a simple “Hello world!” test message. Finally, in Part 3, the two subsystems were integrated: the configuration of `mosquitto.conf` (opening port 1883 and allowing anonymous connections) made the broker reachable from the ESP32 across the Wi-Fi network, and the node was reprogrammed to publish its sensor data over MQTT to the broker. A client on the gateway received the data in real time and printed them as structured JSON objects.

From a learning point of view, the experience showed that the strength of the IoT paradigm lies in the combination of relatively simple components – a microcontroller, a sensor, a single-board computer, a broker – arranged into a layered architecture in which each component has a clear and well-defined role. The use of standard protocols (Wi-Fi, HTTP, MQTT) makes the system modular and scalable: additional nodes can be added simply by configuring them to publish to new topics, without changing the gateway or the existing nodes. The wildcard subscription `system/+/#data` used during the laboratory is in fact a direct demonstration of this scalability, as it would allow a single subscriber to receive data from an arbitrary number of sensor nodes identified by their own ID. Possible future extensions of this work include the addition of multiple nodes, the storage of received data in a local database, and the development of a web dashboard for visual-

ization, which would complete the application layer of the IoT architecture introduced at the beginning of this report.